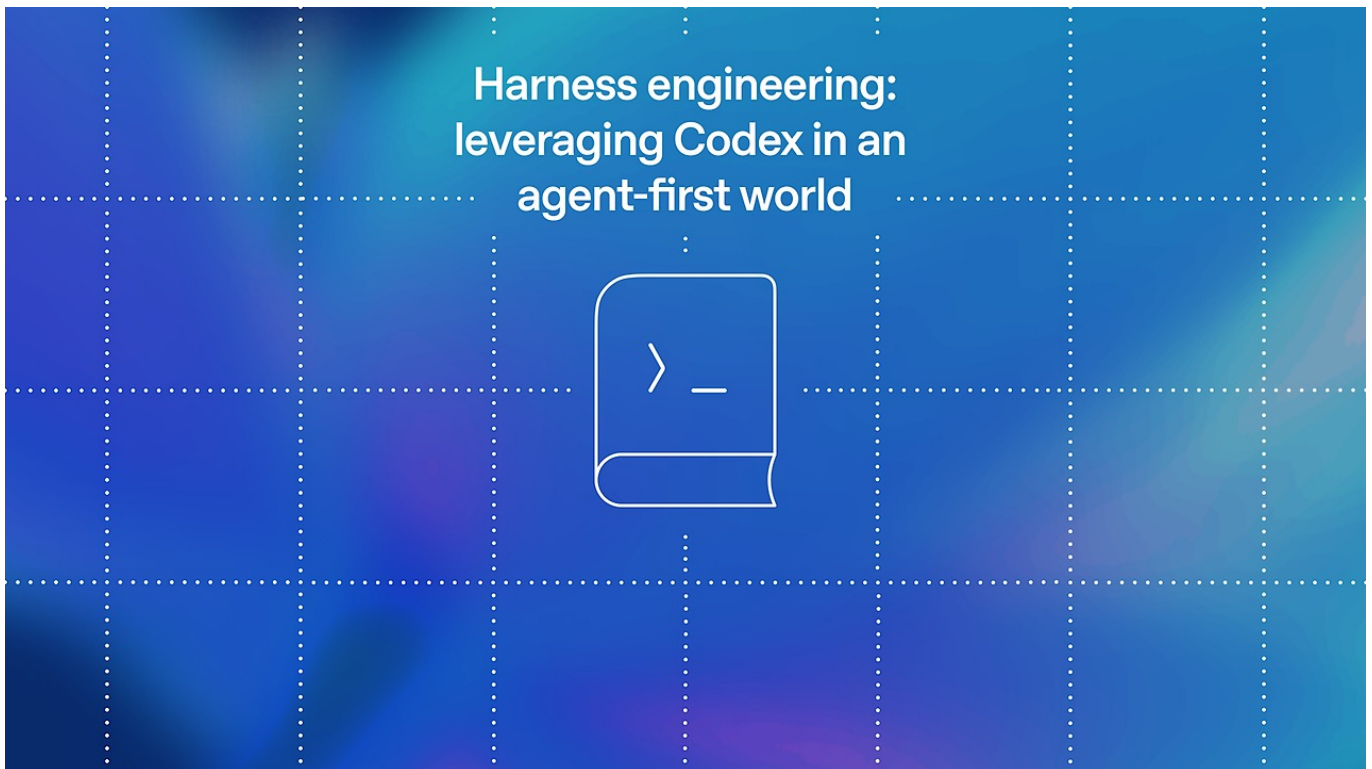




Open AI - harness engineering, skill로 만들어 레포지토리 평가하기

개발/AI 활용 · 2026. 2. 16. 20:52



들어가며

2026년 2월 11일, OpenAI에서 글을 발표했습니다. "[Harness engineering: leveraging Codex in an agent-first world](#)" – 3명의 엔지니어가 코드 0줄에서 100만 줄 규모의 소프트웨어를 만들었다는 내용입니다.

이 글에서의 핵심 메시지는 다음과 같습니다.

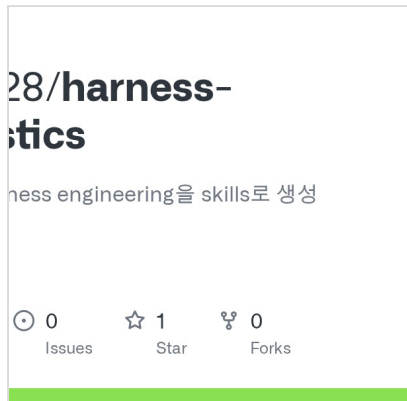
"소프트웨어 엔지니어링 팀의 주된 업무는 더 이상 코드를 작성하는 것이 아니라, 환경을 설계하고, 의도를 명시하며, 에이전트가 신뢰할 수 있는 작업을 수행할 수 있는 피드백 루프를 구축하는 것이다."

이 글을 읽고 분석하면서, 내 프로젝트의 에이전트는 일을 잘 할 수 있는 환경인지 측정하고, 개선

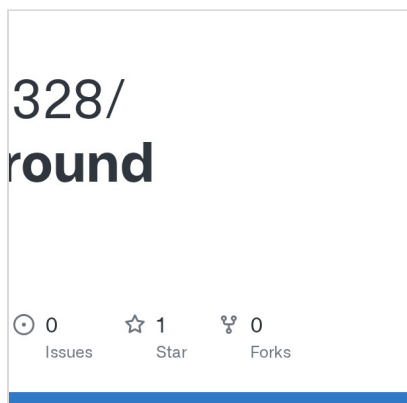
하고 싶다는 생각이 들었습니다.

/harness-diagnostics 코드베이스의 에이전트 친화도를 체계적으로 진단하고 계산하는 Claude Code 스킬을 만들었습니다.

skill 다운 받기 (codex) v.2.1.0

 The image shows a GitHub repository card for 'junh0328/harness-diagnostics'. It features the repository name, a description in Korean and English, and statistics: 0 issues, 1 star, and 0 forks. The card has a green header bar.	GitHub - junh0328/harness-diagnostics Open AI의 harness engineering을 skills로 생성. Contribute to junh0328/harness-diagnostics development by creating an... github.com
--	--

skill 다운 받기 (claude) v.2.1.0

 The image shows a GitHub repository card for 'junh0328/playground'. It features the repository name, a description in Korean and English, and statistics: 0 issues, 1 star, and 0 forks. The card has a blue header bar.	playground/content/skills/harness-dia... Contribute to junh0328/playground development by creating an account on GitHub. github.com
--	--

2026.3.13일 기준 codex는 레포지토리로 분리되었고, claude skill은 해당 레포지토리에 담겨있습니다.

OpenAI가 말하는 Harness Engineering

먼저 원문의 핵심 개념을 짚어보겠습니다.

OpenAI 팀은 Codex 에이전트로만 코드를 작성하면서, **에이전트가 효과적으로 일하기 위한 환경 조건**을 체계적으로 발견해 나갔습니다. 주요 교훈을 정리하면 다음과 같습니다.

1. 지도를 줘라, 매뉴얼을 주지 마라

"Give Codex a map, not a 1,000-page instruction manual."

AGENTS.md를 백과사전이 아닌 **목차(table of contents)**로 취급합니다. 약 100줄의 짧은 진입점이

더 깊은 문서로의 포인터 역할을 합니다.

왜? 컨텍스트는 희소한 자원이기 때문입니다. 거대한 지시 파일은 작업과 코드를 밀어내고, 에이전트가 핵심 제약을 놓치게 만듭니다. 모놀리식 매뉴얼은 즉시 부패하며, 에이전트는 무엇이 여전히 유효한지 구별할 수 없습니다.

2. 불변식을 강제하되, 구현을 마이크로매니징하지 마라

"Enforce invariants, not implementations."

경계에서 데이터 형태를 파싱하도록 요구하되, *어떻게* 하는지는 규정하지 않습니다. 엄격한 아키텍처 모델(Types → Config → Repo → Service → Runtime → UI)을 커스텀 린터와 구조적 테스트로 기계적으로 강제합니다.

3. 레포지토리를 유일한 진실의 원천으로

"Knowledge that lives in Google Docs, chat threads, or people's heads are not accessible to the system."

에이전트의 관점에서, 실행 중 컨텍스트 내에서 접근할 수 없는 것은 **사실상 존재하지 않습니다**. Slack에서 합의한 아키텍처 패턴도 레포지토리에 없으면 에이전트에게는 읽을 수 없습니다.

4. 가비지 컬렉션처럼 지속적으로 정리하라

초기에는 매주 금요일(주의 20%)을 "AI slop" 정리에 사용했지만 확장되지 않았습니다. 대신 **골든 원칙(golden principles)**을 레포지토리에 인코딩하고, 백그라운드 에이전트가 일탈을 스캔하여 리팩토링 PR을 여는 반복 프로세스를 구축했습니다.

원문에서 스킬로: 개념의 체계화

OpenAI의 글은 실전 경험에서 우리나라 통찰이지만, "그래서 내 프로젝트에 어떻게 적용하지?"라는 질문에는 직접 답하지 않습니다. 원문의 교훈은 서사적이고 암묵적입니다. 이를 **측정 가능하고, 반복 실행 가능한 프레임워크**로 변환하는 것이 /harness-diagnostics의 목표입니다.

OpenAI 원문의 교훈	스킬의 대응 원칙
"Map, not Manual"	P2: Map, Not Manual + P5: Progressive Disclosure
"Enforce invariants"	P3: Invariant Enforcement + P4: Convention Over Configuration
"Layered domain architecture"	P6: Layered Architecture + P11: Modularity
"Repository as source of truth"	P9: Knowledge in Repo + P1: Agent Entry Point
"Garbage collection"	P7: Garbage Collection

OpenAI 원문의 교훈	스킬의 대응 원칙
"Agent legibility"	P8: Observability + P12: Self-Documentation
"Reproducible builds"	P10: Reproducibility

원문의 서사에서 12개의 독립 평가 가능한 원칙을 추출하고, 각 원칙에 0~10점의 정량 기준을 부여했습니다. 이렇게 하면 "우리 프로젝트는 에이전트 친화적인가?"라는 질문에 숫자로 답할 수 있습니다.

12가지 Harness 원칙

#	원칙	핵심 질문
P1	Agent Entry Point	명확한 진입점(AGENTS.md/CLAUDE.md)이 존재하는가?
P2	Map, Not Manual	문서가 순차적 매뉴얼이 아닌 탐색 가능한 지도인가?
P3	Invariant Enforcement	도구가 실수를 자동으로 잡는가? (lint, type check, test)
P4	Convention Over Configuration	암묵적 규칙이 아닌 명시적 컨벤션이 있는가?
P5	Progressive Disclosure	필요한 정보를 필요한 시점에 접근할 수 있는가?
P6	Layered Architecture	의존성 방향이 명확하고 강제되는가?
P7	Garbage Collection	낡은 코드/문서가 정기적으로 정리되는가?
P8	Observability	에이전트가 작업 결과를 스스로 검증할 수 있는가?
P9	Knowledge in Repo	지식이 사람의 머릿속이 아닌 저장소에 있는가?
P10	Reproducibility	같은 입력에 같은 결과가 보장되는가?
P11	Modularity	변경의 영향 범위가 예측 가능한가?
P12	Self-Documentation	코드 자체가 의도를 설명하는가?

원칙은 어떻게 측정하나?

각 원칙에는 정량적 측정 지표가 정의되어 있습니다. 예를 들어 **P3 Invariant Enforcement**의 경우:

- pre-commit 또는 CI에서 lint/type-check/test가 자동 실행되는가?
- 위반 시 커밋/PR이 차단되는가?
- 커버리지가 핵심 경로를 포함하는가?

이처럼 주관적 판단이 아닌 확인 가능한 기준으로 점수를 산출합니다. "느낌"이 아니라 "사실"로 평가합니다.

측정 리스트 (codebase-checklist.md)

1. Agent Entry Point (원칙 1) – 15항목

- [] 레포 루트에 `AGENTS.md` 또는 `CLAUDE.md` 파일이 존재하는가
- [] 프로젝트 한 줄 요약이 포함되어 있는가
- [] 기술 스택(언어, 프레임워크, 주요 라이브러리)이 명시되어 있는가
- [] 아키텍처 요약이 포함되어 있는가
- [] 빌드 명령어가 복사-붙여넣기 가능한 형태로 기재되어 있는가
- [] 실행 명령어가 기재되어 있는가
- [] 테스트 실행 명령어가 기재되어 있는가
- [] 단일 파일/모듈 테스트 명령어가 기재되어 있는가
- [] 디렉토리 구조 설명이 포함되어 있는가
- [] 코딩 컨벤션이 기재되어 있는가
- [] 흔한 실수/주의사항(gotchas)이 기재되어 있는가
- [] 관련 문서 링크가 제공되는가
- [] 환경 변수 및 설정 안내가 있는가
- [] 의존성 설치 방법이 명시되어 있는가
- [] 최근 코드 변경사항을 반영하고 있는가 (git log와 비교)

2. 문서 구조 (원칙 2, 5, 9) – 12항목

- [] `docs/` 디렉토리가 존재하는가
- [] `docs/` 내부가 카테고리별로 구조화되어 있는가
- [] ADR(Architecture Decision Records)이 존재하는가
- [] ADR에 결정 배경과 대안이 기록되어 있는가
- [] 개발 가이드가 문서화되어 있는가
- [] API 문서가 존재하는가 (OpenAPI, JSDoc 등)
- [] README.md가 최소한의 정보를 포함하는가
- [] Progressive Disclosure 구조 (요약 → 상세 → 링크)인가
- [] 문서 간 상호 참조(cross-reference)가 있는가
- [] 코드 내 인라인 문서가 "왜(why)"를 설명하는가

- [] CHANGELOG 또는 릴리스 노트가 관리되고 있는가
- [] 문서에 실행 가능한 코드 예제가 포함되어 있는가

3. Invariant 강제 (원칙 3, 4, 10) – 15항목

- [] Linter 설정 파일이 존재하는가 (ESLint, Pylint 등)
- [] Linter 규칙이 프로젝트에 적절한가
- [] Formatter 설정이 존재하는가 (Prettier, Black 등)
- [] Formatter가 자동 적용되는가 (저장 시 또는 commit 시)
- [] Type checking이 설정되어 있는가 (TypeScript strict, mypy 등)
- [] Type checking 엄격도가 적절한가 (strict mode, any 제한)
- [] CI/CD 파이프라인이 존재하는가
- [] CI에서 lint가 실행되는가
- [] CI에서 테스트가 실행되는가
- [] CI에서 type check가 실행되는가
- [] Pre-commit hooks가 설정되어 있는가
- [] 테스트 커버리지 기준이 설정되고 CI에서 강제되는가
- [] Branch protection 규칙이 설정되어 있는가
- [] Import 순서 규칙이 도구로 강제되는가
- [] 보안 검사(의존성 취약점 스캔)가 자동 실행되는가

4. 아키텍처 (원칙 6, 11) – 12항목

- [] 레이어 분리가 되어 있는가 (presentation/domain/infrastructure)
- [] 의존성 방향이 일관적인가 (순환 의존성 없음)
- [] 모듈 경계가 명확한가 (public API 정의)
- [] Import 규칙이 존재하고 강제되는가
- [] API contract가 명시적으로 정의되어 있는가
- [] 설정과 코드가 분리되어 있는가
- [] 공통 유틸리티가 중앙화되어 있는가
- [] 에러 처리 패턴이 프로젝트 전반에서 일관적인가
- [] 데이터 모델이 한 곳에서 정의되어 있는가

- [] 외부 의존성이 추상화 레이어를 통해 접근되는가
- [] 테스트 구조가 소스 구조와 대칭적인가
- [] 순환 의존성이 없는가 (도구로 검증)

5. Repository as Source of Truth (원칙 9, 12) – 10항목

- [] 기술적 결정이 레포 내(ADR, PR)에 기록되는가
- [] "왜 그렇게 구현했는지" 배경 정보가 코드/commit/PR에 있는가
- [] 계획/로드맵이 레포 내에서 관리되는가 (issues, ROADMAP.md)
- [] TODO/FIXME가 이슈와 연결되고 주기적으로 정리되는가
- [] 변수명, 함수명만으로 의도를 파악할 수 있는가
- [] 온보딩에 필요한 모든 정보가 레포에 있는가
- [] 외부 의존성 선택 이유가 기록되어 있는가
- [] 레포만으로 개발 환경을 처음부터 셋업할 수 있는가
- [] Commit 메시지가 변경의 의도를 충분히 설명하는가
- [] PR 템플릿이 존재하는가

6. 운영/유지보수 (원칙 7, 8) – 10항목

- [] Dead code 탐지 도구가 설정되어 있는가 (knip, ts-prune 등)
- [] Dead code가 주기적으로 제거되고 있는가
- [] 의존성이 주기적으로 업데이트되는가 (Dependabot, Renovate)
- [] Stale/deprecated 의존성이 식별되고 있는가
- [] 구조화된 로깅이 적용되어 있는가
- [] 에러 모니터링 도구가 연동되어 있는가 (Sentry, Datadog)
- [] Health check 엔드포인트가 있는가
- [] 불필요한 설정 파일이 정리되어 있는가
- [] Docker 이미지가 최적화되어 있는가
- [] Flaky test가 관리되고 있는가

7. Agent 가독성 (원칙 2, 5, 12) – 10항목

- [] 파일명과 디렉토리 구조만으로 코드 위치를 예측할 수 있는가
- [] 개별 파일이 적절한 크기를 유지하는가 (300줄 이하)
- [] 함수/메서드가 적절한 크기인가 (50줄 이하)
- [] 네이밍 컨벤션이 프로젝트 전반에서 일관적인가
- [] 애플리케이션 entry point가 명확히 식별 가능한가
- [] 모듈의 public API가 명확히 정의되어 있는가
- [] 매직 넘버/문자열이 상수로 추출되어 있는가
- [] 조건문/반복문 중첩이 과도하지 않은가 (3단계 이하)
- [] 같은 유형의 파일이 일관된 내부 구조를 따르는가
- [] 주석이 "what"이 아닌 "why"를 설명하는가

결과 요약

카테고리	충족	전체	비율
1. Agent Entry Point	/15	15	%
2. 문서 구조	/12	12	%
3. Invariant 강제	/15	15	%
4. 아키텍처	/12	12	%
5. Source of Truth	/10	10	%
6. 운영/유지보수	/10	10	%
7. Agent 가독성	/10	10	%
합계 **/84** **84** **%**			

우선순위 가이드

1. **카테고리 1** (Agent Entry Point) – 에이전트 이해의 출발점
2. **카테고리 3** (Invariant 강제) – 에이전트 실수를 잡는 안전망
3. **카테고리 4** (아키텍처) – 올바른 위치에 코드 작성
4. **카테고리 7** (Agent 가독성) – 빠르고 정확한 코드 이해
5. **카테고리 2** (문서 구조) – 심층 맥락 전달
6. **카테고리 5** (Source of Truth) – 장기적 지식 관리

5단계 성숙도 모델

12가지 원칙의 점수는 4개 차원으로 묶여 가중 평균을 통해 최종 성숙도 레벨로 변환됩니다.

4개 차원과 가중치

차원	포함 원칙	가중치
A. 문서 & 탐색	P1, P2, P5, P12	30%
B. 검증 & 일관성	P3, P4, P10	30%
C. 아키텍처 & 지식	P6, P9, P11	20%
D. 운영 & 유지보수	P7, P8	20%

문서와 검증에 각 30%의 높은 비중을 둔 이유는 OpenAI 원문에서도 강조하는 부분입니다. 에이전트가 작업을 시작하려면 **무엇을 해야 하는지 알아야 하고(문서), 올바르게 했는지 확인할 수 있어야 합니다(검증)**. 이 두 가지가 없으면 나머지 원칙은 의미가 없습니다.

5단계 레벨

L1: None (0-19) – 에이전트 협업을 고려하지 않은 상태
 L2: Basic (20-39) – 최소한의 문서만 존재
 L3: Structured (40-59) – 체계적 구조, 부분 자동화
 L4: Optimized (60-79) – 높은 자동화, 낮은 drift
 L5: Autonomous (80-100) – 에이전트가 독립적으로 기여 가능

OpenAI가 도달한 수준은 L5에 가깝습니다. Codex가 새 기능을 엔드투엔드로 구동하고, 판단이 필요할 때만 인간에게 에스컬레이션하는 수준입니다. 대부분의 프로젝트는 L2~L3에서 시작하게 됩니다.

산출 공식:

$$\text{Score} = (\text{DimA} \times 0.3 + \text{DimB} \times 0.3 + \text{DimC} \times 0.2 + \text{DimD} \times 0.2) \times 10$$

3가지 운영 모드

/harness-diagnostics는 프로젝트의 상황에 따라 3가지 모드를 제공합니다.

1. Setup 모드 – "새 프로젝트에 에이전트 환경을 구축할 때"

OpenAI 팀이 빈 레포지토리에서 시작했듯이, 에이전트 협업을 처음 도입하는 프로젝트에서 사용합니다.

워크플로우:

1. 현재 상태 스캔 (기술 스택, 기존 문서, 도구 확인)
2. 갭 분석 (12원칙 대비 누락된 요소 식별)
3. 실행 계획 생성 (즉시/단기/중기)
4. 리포트 출력

기술 스택(Node.js, Python, Go 등)을 자동 감지하여 스택별 맞춤 추천을 제공합니다. 예를 들어 Node.js 프로젝트라면 ESLint + TypeScript strict mode + Vitest 같은 구체적인 도구를 추천합니다.

2. Audit 모드 – "기존 프로젝트를 종합 평가할 때"

이미 운영 중인 프로젝트의 에이전트 친화도를 종합 진단합니다. 코드베이스는 84항목, 스킬은 40항목의 체크리스트로 평가합니다.

리포트 예시 (가상 프로젝트 "TaskFlow"):

```
=====
harness-diagnostics Audit Report
대상: TaskFlow
날짜: 2026-02-13
=====

종합 점수: 58/100 → Level 3 (Structured)

차원별 점수:
  A. 문서 & 탐색      6.5/10
  B. 검증 & 일관성    7.0/10
  C. 아키텍처 & 지식   5.0/10
  D. 운영 & 유지보수   4.0/10

Quick Wins (1주 내 실행 가능):
```

1. AGENTS.md 생성 (P1: +3점 예상)
2. pre-commit hook에 type-check 추가 (P3: +1점 예상)
3. docs/conventions.md 작성 (P4: +2점 예상)

Quick Win은 적은 노력으로 큰 점수 향상을 기대할 수 있는 항목입니다. AGENTS.md 하나만 만들어도 P1 점수가 크게 올라갑니다.

3. Maintenance 모드 – "시간이 지난 프로젝트를 점검할 때"

OpenAI가 "가비지 컬렉션"이라 부른 것의 진단 버전입니다. 프로젝트가 성장하면서 발생하는 **drift**(문서와 실제 코드의 괴리)를 감지합니다.

Drift의 4가지 유형:

유형	설명	예시
Doc Drift	문서가 코드 변경을 반영하지 않음	API 엔드포인트 변경 후 문서 미갱신
Config Drift	설정이 실제 환경과 불일치	.env.example에 새 변수 누락
Convention Drift	새 코드가 기존 컨벤션 미준수	네이밍 규칙 위반
Architecture Drift	의존성 방향 위반	레이어 간 순환 참조 발생

OpenAI 원문에서 경고한 것처럼, "에이전트는 레포지토리에 이미 존재하는 패턴을 복제합니다 – 불균일하거나 차선의 패턴도 포함하여." drift를 방지하면 에이전트가 나쁜 패턴을 복제하는 악순환이 시작됩니다.

스킬 구조와 설계 철학

파일 구조

```
harness-diagnostics/  
├─ SKILL.md # 진입점 (v1.2.0)  
├─ references/  
│   └─ principles.md # 12원칙 정의 + 점수 기준  
│   └─ maturity-framework.md # 5단계 성숙도 모델  
│   └─ codebase-checklist.md # 코드베이스 평가 84항목  
│   └─ skill-checklist.md # 스킬 평가 40항목  
│   └─ report-template.md # 리포트 출력 형식  
│   └─ setup-workflow.md # Setup 모드 워크플로우
```

```
| └─ audit-workflow.md          # Audit 모드 워크플로우
| └─ maintenance-workflow.md   # Maintenance 모드 워크플로우
└─ examples/                   # 참조 예시
    └─ sample-agents-md.md      # AGENTS.md 예시
    └─ sample-report.md         # Audit 리포트 예시
    └─ sample-docs-structure.md # docs/ 디렉토리 구조 가이드
```

설계 원칙: 스킬 자체가 원칙을 실천한다

이 스킬을 만들 때 가장 중요하게 생각한 것이 있습니다. **진단 도구 자체가 자신이 제시하는 원칙을 따르지 못한다면 신뢰할 수 없습니다.** 이를 위해 4가지 설계 원칙을 적용했습니다.

1. Read-Only 진단

코드를 분석만 하고 **절대 수정하지 않습니다.** 모든 개선 사항은 리포트 형태로 제안됩니다. OpenAI가 "불변식을 강제하되 구현을 마이크로매니징하지 않는다"고 한 것과 같은 맥락입니다. 무엇이 문제인지 알려주되, 어떻게 고칠지는 개발자의 판단에 맡깁니다.

2. 자기 참조(Self-Referential)

/harness-diagnostics 스킬 자체를 진단 대상으로 지정할 수 있습니다. 스킬 체크리스트 40항목에 대해 자체 평가 점수는 **40/40 (100%)**입니다.

3. Map, Not Manual 구조

SKILL.md는 약 200줄의 지도 역할을 합니다. 모든 내용을 한 파일에 넣지 않고, 현재 모드에 필요한 참조 문서로 안내합니다. OpenAI가 AGENTS.md를 100줄의 목차로 취급한 것과 같은 접근입니다.

4. 순환 참조 없는 단방향 의존성

```
SKILL.md (진입점)
└─ workflows (Setup/Audit/Maintenance)
    └─ report-template
└─ principles + maturity-framework
└─ checklists (codebase/skill)
└─ examples (리프 노트)
```

OpenAI의 "Layered Architecture"를 파일 구조 수준에서 실천한 것입니다. 하위 파일이 상위 파일을 참조하지 않으며, 의존성이 항상 아래로만 흐릅니다.

사용 예시

Claude Code에서 다음과 같이 실행합니다.

```
/harness-diagnostics
```

실행하면 대상과 모드를 선택하는 대화가 시작됩니다.

```
? 진단 대상을 선택하세요
  ○ 코드베이스 (현재 디렉토리)
  ○ Claude Code 스킴
  ○ Self (harness-diagnostics 자체)

? 진단 모드를 선택하세요
  ○ Setup    - 새 프로젝트 환경 구축
  ○ Audit    - 종합 평가
  ○ Maintenance - drift 점검
```

선택 후 프로젝트를 자동 분석하고, 원칙별 점수와 구체적인 개선 방안이 담긴 마크다운 리포트를 생성합니다.

...

실전 활용 시나리오

시나리오 1: 팀에 AI 에이전트 도입 준비

```
/harness-diagnostics → 코드베이스 → Setup
```

기존 프로젝트에 AGENTS.md를 비롯한 에이전트 협업 환경을 처음 세팅할 때 적합합니다. 기술 스택을 자동 감지하고, 우선순위에 따른 실행 계획을 제시합니다.

시나리오 2: 분기별 에이전트 환경 점검

```
/harness-diagnostics → 코드베이스 → Maintenance
```

3개월간 프로젝트가 발전하면서 문서와 코드 사이에 생긴 괴리를 감지합니다. git 히스토리를 분석하여 어떤 영역에서 drift가 발생했는지 알려줍니다.

시나리오 3: 직접 만든 Claude Code 스킬 품질 검증

/harness-diagnostics → 스킬 → Audit

직접 만든 스킬이 40항목의 품질 기준을 충족하는지 점검합니다. 파일 구조, SKILL.md 품질, frontmatter, 참조 문서 등을 체계적으로 평가합니다.

마무리

OpenAI의 Harness Engineering 글이 던진 핵심 메시지는 다음과 같습니다. "소프트웨어 구축에는 여전히 규율이 필요하지만, 그 규율은 코드보다 스캐폴딩에서 더 드러난다."

1. 에이전트 친화도는 측정 가능합니다 – 12원칙, 84항목 체크리스트, 정량 점수로 객관적으로 평가할 수 있습니다.
2. 성숙도는 단계적으로 높일 수 있습니다 – L1부터 L5까지, Quick Win부터 시작하여 점진적으로 개선할 수 있습니다.
3. 환경은 지속적으로 관리해야 합니다 – OpenAI도 "가비지 컬렉션"이라 부른 것처럼, 한번 세팅하고 끝이 아닙니다.

참고 자료

- [Harness engineering: leveraging Codex in an agent-first world | OpenAI](#) – 이 스킬의 기반이 된 OpenAI 원문
- [Unlocking the Codex harness: how we built the App Server | OpenAI](#) – Harness Engineering 후속 글
- [Anthropic - Claude Code 공식 문서](#)
- [Claude Code Skills 작성 가이드](#)

♡ 10 📌 🗨

구독하기



'개발 > AI 활용' 카테고리의 다른 글

[A practical guide to building agents: OpenAI 에이전트 구축 실전 가이드](#) (0)

2026.03.16

Harness Engineering 실전: 프로젝트 도입 및 재평가 루프 만들기 (0)	2026.02.24
Text-to-SQL 딥다이브 – 자연어로 데이터베이스를 조회하는 앱 만들기 (0)	2026.02.10
SSE 트러블 슈팅 가이드 - 실시간 스트리밍 메시지 순서가 뒤바뀌어 보여지는 문제 해결하기 (0)	2026.02.09
만들면서 배우는 챗봇 고도화: multi_provider 전략 (0)	2026.02.03

TAG Harness engineering , OpenAI

관련글

[관련글 더보기](#)

A practical guide to building agents: OpenAI 에이전트 구축 실전 가이드

Harness Engineering 실전: 프로젝트 도입 및 재평가 루프 만들기

Text-to-SQL 딥다이브 – 자연어로 데이터베이스를 조회하는 앱 만들기

SSE 트러블 슈팅 가이드 - 실시간 스트리밍 메시지 순서가 뒤바뀌어 보여지는 ...

댓글 0

프론트엔드 개발자의 기억 저장소

프론트엔드 개발자로 살아 남기 위한 기억 저장소입니다. (2022.01 ~)

구독하기 +

댓글을 입력해주세요.

이름

비밀번호

☐비밀글

등록

Designed by Tistory.